Agentic架构 3：推理+行动（ReAct）

> **推理+行动（ReAct）** —— 弥合了**简单工具使用与复杂多步骤问题解决**之间的鸿沟。核心创新在于它使 Agent 能够**动态地推理问题**、基于**推理**采取行动、观察结果，然后再次进行推理。这种模式将 Agent 从一个静态的工具调用者转变为一个**自适应的问题解决者**。

定义

ReAct 架构是一种设计模式，其中 Agent 将**推理步骤与行动交错进行**。Agent 不是提前规划所有步骤，而是针对其下一步**行动**生成一个**想法**，采取**行动（如调用工具）**，**观察结果**，然后利用这些新信息生成下一个想法和行动。这就创建了一个**动态且自适应的循环**。

高层工作流

1. **接收目标：** Agent 被赋予一个复杂任务。
2. **思考（推理）：** Agent 生成一个内部想法，例如："要回答这个问题，我首先需要找到信息 X。"
3. **行动：** 基于其想法，Agent 执行一个行动，通常是**调用一个工具**（例如，`search_api('X')`）。
4. **观察：** Agent 接收来自工具的结果。
5. **重复：** Agent **将观察结果整合到其上下文中**，并返回到第 2 步，生成新的想法（例如，"好的，现在我有了 X，我需要用它来找到 Y。"）。这个循环持续进行，直到总体目标达成。

适用场景 / 应用

- **多跳问答：** 当回答一个问题需要**按顺序查找多条信息**时（例如，"制造 iPhone 的公司其 CEO 是谁？"）。
- **网页导航与研究：** Agent 可以搜索起点，阅读结果，然后根据所学内容决定新的搜索查询。
- **交互式工作流：** 任何环境是**动态**的、无法预先知道完整解决方案路径的任务。

优势与劣势

- **优势：**
  - **自适应与动态性：** 能够根据新信息**即时调整计划**。
  - **处理复杂性：** 擅长处理需要串联**多个依赖步骤**的问题。
- **劣势：**
  - **更高的延迟与成本：** 涉及**多次连续的 LLM 调用**，因此比单次方法更慢、成本更高。
  - **循环风险：** 缺乏良好引导的 Agent 可能会陷入**重复、无效的思考和行动循环**中。

阶段 0：基础与环境搭建

从标准的环境搭建流程开始：安装库，并为 Nebius、LangSmith 以及 Tavily 网络搜索工具配置 API 密钥。

步骤 0.1：安装核心库

为本项目安装标准套件的库

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
tavily-python
```

步骤 0.2：导入库与配置密钥

导入必要的模块，并从 `.env` 文件中加载 API 密钥。

需要执行的操作： 在此目录下创建一个 `.env` 文件，并填入密钥：

```
NEBIUS_API_KEY="your_nebius_api_key_here"
LANGCHAIN_API_KEY="your_langsmith_api_key_here"
TAVILY_API_KEY="your_tavily_api_key_here"
```

```python
import os
from typing import Annotated
from dotenv import load_dotenv

# LangChain 组件
from langchain_nebius import ChatNebius
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_core.messages import BaseMessage
from pydantic import BaseModel, Field

# LangGraph 组件
from langgraph.graph import StateGraph, END
from langgraph.graph.message import AnyMessage, add_messages
from langgraph.prebuilt import ToolNode, tools_condition

# 美化打印输出
from rich.console import Console
from rich.markdown import Markdown

# --- API 密钥与追踪配置 ---
load_dotenv()  # 加载 .env 文件中的环境变量

# 配置 LangSmith 追踪功能
os.environ["LANGCHAIN_TRACING_V2"] = "true"  # 启用 LangSmith 追踪 v2 版本
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - ReAct (Nebius)"
# 设置追踪项目名称

# 校验必需的环境变量是否已配置
for key in ["NEBIUS_API_KEY", "LANGCHAIN_API_KEY", "TAVILY_API_KEY"]:
    if not os.environ.get(key):
        print(f"{key} not found. Please create a .env file and set it.")

print("Environment variables loaded and tracing is set up.")
```

阶段 1：基础方法——单次工具使用者

首先看看没有ReAct框架时的情况。构建一个 **“基础”Agent**，它可以**使用工具**，但只能使用一次。它会**分析**用户的查询，进行一次**工具调用**，然后尝试基于这一条信息制定最终答案。

步骤 1.1：构建基础 Agent

定义相同的工具和 LLM，将它们连接到一个**简单的线性图**结构中。Agent 只有一次**调用工具**的机会，然后工作流就结束了。这里没有循环。

```
from typing import TypedDict

# 初始化 Rich 控制台，用于美化打印输出
console = Console()

# 定义智能体状态结构，用于管理对话历史
class AgentState(TypedDict):
    """
    智能体状态结构，用于管理对话历史。

    使用 Annotated 类型与 add_messages 函数配合，实现消息列表的自动合并更新。
    当向状态中添加新消息时，LangGraph 会自动将新消息追加到现有消息列表末尾。
    """
    messages: Annotated[list[AnyMessage], add_messages]  # 对话消息列表，支持
自动追加更新

# 定义工具和大语言模型
# 初始化 Tavily 搜索工具，设置最大返回结果数并指定工具名称
search_tool = TavilySearchResults(max_results=2, name="web_search")

# 初始化 Nebius 大语言模型，使用 Llama 3.1 8B 指令微调版本
# 设置 temperature=0 使输出结果更具确定性和一致性
llm = ChatNebius(model="meta-llama/Meta-Llama-3.1-8B-Instruct",
temperature=0)

# 将工具绑定到 LLM，使模型具备工具感知能力
llm_with_tools = llm.bind_tools([search_tool])

# 定义基础智能体节点
def basic_agent_node(state: AgentState):
    """基础智能体节点，调用 LLM 决定下一步行动，限制为单次工具调用后必须回答。"""
    console.print("--- BASIC AGENT: Thinking... ---")  # 在控制台输出智能体思
考状态提示

    # 注意：通过系统提示词引导模型在一次工具调用后直接给出最终答案
    system_prompt = "You are a helpful assistant. You have access to a web
search tool. Answer the user's question based on the tool's results. You
must provide a final answer after one tool call."

    # 将系统提示词与现有消息合并，构建完整的消息列表
    messages = [("system", system_prompt)] + state["messages"]
```

```
    # 调用已绑定工具的 LLM，获取模型响应
    response = llm_with_tools.invoke(messages)

    # 返回响应消息，LangGraph 会自动将其追加到状态的消息列表中
    return {"messages": [response]}

# 定义基础的线性工作流图
basic_graph_builder = StateGraph(AgentState)
# 向图中添加节点
basic_graph_builder.add_node("agent", basic_agent_node)  # 智能体节点
basic_graph_builder.add_node("tools", ToolNode([search_tool]))  # 工具执行节
点
# 设置入口节点为智能体节点
basic_graph_builder.set_entry_point("agent")

# 添加条件边：从智能体节点出发，根据 tools_condition 路由函数决定下一跳
# - 如果 LLM 请求调用工具，则路由到 "tools" 节点
# - 如果 LLM 未请求工具调用，则结束流程
basic_graph_builder.add_conditional_edges(
    "agent",
    tools_condition,  # LangGraph 预置的条件路由函数，检查消息中的 tool_calls
    {"tools": "tools", "__end__": "__end__"}  # 路由映射表
)

# 添加从工具节点到结束节点的边，这意味着工具执行后流程立即结束，不支持多轮工具调用
basic_graph_builder.add_edge("tools", END)
# 编译图，生成可执行的应用实例
basic_tool_agent_app = basic_graph_builder.compile()

print("Basic single-shot tool-using agent compiled successfully.")
```

步骤 1.2：在多步骤问题上测试基础 Agent

给基础 Agent 一个需要**多个依赖步骤**才能解决的问题。这将暴露出它的根本弱点。

```
# 定义多步查询问题，需要智能体进行多轮推理和信息整合
# 该问题涉及两个子任务：
# 1. 找出电影"Dune"的制作公司的现任 CEO
# 2. 查询该公司最近一部电影的预算
multi_step_query = "Who is the current CEO of the company that created the
sci-fi movie 'Dune', and what was the budget for that company's most recent
film?"

# 在控制台以黄色粗体显示测试信息，表明正在测试基础智能体在多步查询上的表现
console.print(f"[bold yellow]Testing BASIC agent on a multi-step query:
[/bold yellow] '{multi_step_query}'\n")

# 调用基础工具智能体应用，传入用户查询消息
# 基础智能体设计为单次工具调用后直接回答，对于需要多步推理的复杂问题可能表现不佳
basic_agent_output = basic_tool_agent_app.invoke({"messages": [("user",
multi_step_query)]})
```

```
# 输出最终结果的标题（红色粗体）
console.print("\n--- [bold red]Final Output from Basic Agent[/bold red] ---
")

# 获取最后一条消息的内容（即智能体的最终回答）
# 使用 Markdown 格式渲染，支持富文本显示
console.print(Markdown(basic_agent_output['messages'][-1].content))
```

阶段 2：进阶方法——实现 ReAct

构建真正的 **ReAct Agent**。核心区别在于图结构的改变：我们将**引入一个循环**，允许 Agent 反复进行**思考、行动和观察**。

步骤 2.1：构建 ReAct Agent 图结构

定义**节点以及关键的路径函数**，用于创建 `思考 → 行动` **的循环**。关键的架构变化在于，将 `tool_node` 的输出**路由回** `agent_node` 的边，这使得 Agent 能够**看到结果并决定下一步行动**。

```
def react_agent_node(state: AgentState):
    """
    ReAct 智能体节点，负责调用 LLM 进行推理和决策。
    该节点与基础智能体节点的区别在于，它不强制要求一次工具调用后立即结束，而是允许 LLM
在获取工具结果后继续推理，实现多轮思考-行动循环。
    """
    console.print("--- REACT AGENT: Thinking... ---")  # 在控制台输出智能体思
考状态提示
    # 调用已绑定工具的 LLM，传入当前消息列表，获取模型响应
    response = llm_with_tools.invoke(state["messages"])
    # 返回响应消息，LangGraph 会自动将其追加到状态的消息列表中
    return {"messages": [response]}

# ToolNode 与之前相同，负责执行工具调用
react_tool_node = ToolNode([search_tool])

# 路由函数逻辑与之前相同，但返回值使用 "tools" 而非 "call_tool" 以保持命名一致性
def react_router(state: AgentState):
    """
    路由函数，根据智能体的最后一条消息决定下一步流程走向。
    检查最新响应中是否包含工具调用请求：
    - 如果有工具调用，则路由到 "tools" 节点
    - 如果没有工具调用，说明智能体已给出最终答案，结束流程
    """
    # 获取状态中最新的一条消息（即智能体的最近一次响应）
    last_message = state["messages"][-1]

    # 检查消息中是否包含工具调用请求
    if last_message.tool_calls:
        console.print("--- ROUTER: Decision is to call a tool. ---")
        return "tools"  # 返回工具节点标识
```

```
    else:
        console.print("--- ROUTER: Decision is to finish. ---")
        return "__end__"  # 返回结束标识

# 定义包含循环的 ReAct 工作流图
react_graph_builder = StateGraph(AgentState)
# 向图中添加节点
react_graph_builder.add_node("agent", react_agent_node)  # ReAct 智能体节点
react_graph_builder.add_node("tools", react_tool_node)   # 工具执行节点
# 设置入口节点为智能体节点
react_graph_builder.set_entry_point("agent")

# 添加条件边：从智能体节点出发，根据路由函数的返回值决定下一跳
react_graph_builder.add_conditional_edges(
    "agent",
    react_router,  # 自定义路由函数
    {"tools": "tools", "__end__": "__end__"}  # 路由映射表
)
# 这是 ReAct 模式与基础模式的关键区别：
# 从工具节点返回智能体节点，形成循环，支持多轮思考-行动迭代，这使得智能体能够进行多步推理，逐步获取信息并整合答案
react_graph_builder.add_edge("tools", "agent")

# 编译图，生成可执行的应用实例
react_agent_app = react_graph_builder.compile()
print("ReAct agent compiled successfully with a reasoning loop.")
```

阶段 3：正面比较

用新的 ReAct Agent 运行相同的复杂查询，并观察其过程和最终输出的差异。

步骤 3.1：在多步骤问题上测试 ReAct Agent

使用**相同的多步骤查询**调用 ReAct Agent，并流式传输输出以观察其迭代推理过程。

```
# 在控制台以绿色粗体显示测试信息，表明正在使用相同的多步查询测试 ReAct 智能体
console.print(f"[bold green]Testing ReAct agent on the same multi-step query:[/bold green] '{multi_step_query}'\n")

# 初始化变量，用于存储最终输出状态
final_react_output = None

# 流式执行 ReAct 智能体工作流，以 "values" 模式逐个接收状态快照
# 这允许观察智能体在多轮思考-行动循环中的中间状态
for chunk in react_agent_app.stream({"messages": [("user", multi_step_query)]}, stream_mode="values"):
    # 每次迭代更新最终状态，循环结束后 final_react_output 为最后一步的状态
    final_react_output = chunk

    # 输出当前状态的标题（紫色粗体），便于观察执行过程中的每一步
```

```
    console.print(f"--- [bold purple]Current State[/bold purple] ---")

    # 获取当前状态快照中的最后一条消息（即最新响应）
    # 使用 pretty_print() 方法以美观格式打印消息内容，便于调试和理解智能体行为
    chunk['messages'][-1].pretty_print()

    # 在每条消息后打印空行分隔，增强可读性
    console.print("\n")

# 输出最终结果的标题（绿色粗体）
console.print("\n--- [bold green]Final Output from ReAct Agent[/bold green]
---")

# 获取最终状态中的最后一条消息内容（即智能体的最终回答）
# 使用 Markdown 格式渲染，支持富文本显示
console.print(Markdown(final_react_output['messages'][-1].content))
```

> **关于输出的讨论：** 执行轨迹展示了一个完全不同且智能得多的过程。 Agent进行逐步推理：
>
> 1. **思考 1：** 它首先推理出需要识别《沙丘》的制作公司。
> 2. **行动 1：** 它使用类似"production company for Dune movie"的查询调用 `web_search` 工具。
> 3. **观察 1：** 它收到结果："Legendary Entertainment"。
> 4. **思考 2：** 现在，**结合新信息**，它**推理**出需要 Legendary Entertainment 的 CEO。
> 5. **行动 2：** 它再次调用 `web_search`，查询类似"CEO of Legendary Entertainment"。
> 6. ……以此类推，直到收集到所有必要的信息片段。
> 7. **综合：** 最后，它**将所有收集到的事实整合成一个完整且准确的答案**。
>
> 这清楚地证明了 ReAct 模式对于任何不是简单单步查询的任务都具有优越性。

阶段 4：定量评估

为了使比较更加规范化，使用 LLM 作为评判者，对**基础 Agent 和 ReAct Agent** 的最终输出在完成任务能力方面进行评分。

```
class TaskEvaluation(BaseModel):
    """用于评估智能体完成任务能力的结构。"""
    task_completion_score: int = Field(description="任务完成度评分，1-10分（智
能体是否成功完成用户请求的所有部分）。")
    reasoning_quality_score: int = Field(description="推理质量评分，1-10分（智
能体展示的逻辑流程和推理过程质量）。")
    justification: str = Field(description="各项评分的简要说明。")

# 将基础 LLM 包装为结构化输出模型，确保输出符合 TaskEvaluation 的结构定义
judge_llm = llm.with_structured_output(TaskEvaluation)

def evaluate_agent_output(query: str, agent_output: dict):
    """
    评估智能体在给定任务上的表现。
    参数:query: 用户原始查询
        agent_output: 智能体执行后的完整输出状态（包含消息历史）
```

```
    返回:TaskEvaluation: 包含评分和说明的评估结果
    """
    # 重构完整的对话轨迹，提取每条消息的类型和内容
    # 这提供了智能体完整的思考-行动过程，便于评估其推理质量
    trace = "\n".join([f"{m.type}: {m.content}" for m in
agent_output['messages']])

    # 构建评估提示词，要求模型扮演 AI 智能体评审专家角色
    prompt = f"""You are an expert judge of AI agents. Evaluate the
following agent's performance on the given task on a scale of 1-10. A score
of 10 means the task was completed perfectly. A score of 1 means complete
failure.

    **User's Task:**
    {query}

    **Full Agent Conversation Trace:**
    \`\`\`
    {trace}
    \`\`\`
    """
    # 调用 LLM 生成评估结果
    return judge_llm.invoke(prompt)

# 输出基础智能体评估的标题
console.print("--- Evaluating Basic Agent's Output ---")

# 对基础智能体的输出进行评估
basic_agent_evaluation = evaluate_agent_output(multi_step_query,
basic_agent_output)

# 使用 model_dump() 将 Pydantic 模型序列化为字典格式并打印
console.print(basic_agent_evaluation.model_dump())

# 输出 ReAct 智能体评估的标题
console.print("\n--- Evaluating ReAct Agent's Output ---")

# 对 ReAct 智能体的输出进行评估
react_agent_evaluation = evaluate_agent_output(multi_step_query,
final_react_output)

# 使用 model_dump() 将 Pydantic 模型序列化为字典格式并打印
console.print(react_agent_evaluation.model_dump())
```

> **关于输出的讨论：** LLM 作为评判者给出的定量评分使差异一目了然。
>
> - **基础 Agent** 的 `task_completion_score`（任务完成度评分）非常低，因为它未能收集所有所需信息。其 `reasoning_quality_score`（推理质量评分）也很低，因为其过程存在缺陷且不完整。
> - 相比之下，**ReAct Agent** 获得了近乎完美的分数。评判者认为**其迭代过程使其能够成功完成复杂任务的所有部分**。

> 这种正面比较和评估为 ReAct 架构的价值提供了确凿的证据。它是解锁 Agent 处理需要动态适应的复杂多跳问题能力的关键。

结论

在本笔记中，我们不仅实现了 **ReAct** 架构，还证明了它相对于更基础的单次方法的**明显优越性**。通过构建一个允许 Agent **在推理和行动之间循环迭代**的工作流，我们使其能够解决原本难以处理的**复杂多步骤**问题。

观察行动结果并利用这些信息来指导下一步行动的能力，是智能行为的基本组成部分。ReAct 模式提供了一种简单而极其有效的方法，将这种能力构建到我们的 AI Agent 中，使其**更加强大**、**更具适应性**，并且对现实世界的任务更加有用。